

경북대학교 공학석사학위논문

# 강화학습을 이용한 eBPF 커널 신호 기반 MQTT 꼬리 지연 제어에 관한 연구

대학원 컴퓨터학부

정진욱

2025년 10월

경북대학교 대학원

강화학습을 이용한 eBPF 커널 실험  
MOTI 포리 지연 제어에 관한 연구

(공학  
위학  
논석  
무사)

정

진

욱

2025

# 강화학습을 이용한 eBPF 커널 신호 기반 MQTT 꼬리 지연 제어에 관한 연구

이 논문을 공학석사 학위논문으로 제출함

대학원 컴퓨터학부

정진욱

지도교수 권영우

정진욱의 공학석사 학위논문을 인준함

2025년 12월

위원장 \_\_\_\_\_ 권영우 (인)

\_\_\_\_\_ 고석주 (인)

\_\_\_\_\_ 백호기 (인)

경북대학교 대학원위원회

# 목차

|                                   |    |
|-----------------------------------|----|
| Abstract in Korean                | 5  |
| 1 서론                              | 2  |
| 1.1 연구 배경: IoT 엣지 컴퓨팅과 MQTT의 부상   | 2  |
| 1.2 문제 정의: 실시간 IoT 시스템의 꼬리 지연 시간  | 3  |
| 1.3 기존 접근법과 본 연구의 출발점             | 4  |
| 1.4 연구 목표 및 제안 방법: 강화학습 기반의 자율 제어 | 4  |
| 1.5 본 논문의 기여                      | 5  |
| 1.6 논문의 구성                        | 6  |
| 2 관련연구                            | 8  |
| 2.1 MQTT 프로토콜과 IoT 지연 문제          | 8  |
| 2.2 꼬리 지연(tail latency)과 제어 필요성   | 10 |
| 2.3 eBPF를 활용한 커널 신호 관측            | 11 |
| 2.4 강화학습을 이용한 네트워크 제어 동향          | 13 |
| 3 eBPF 기반 실시간 커널 관측               | 14 |
| 3.1 꼬리 지연 제어의 중요성                 | 15 |
| 3.2 eBPF 커널 관측의 당위성               | 16 |
| 3.3 eBPF 기술 개요                    | 18 |
| 3.4 핵심 관측 신호                      | 19 |
| 3.5 eBPF 프로그램 구현                  | 20 |
| 3.5.1 데이터 구조                      | 20 |
| 3.5.2 커널 프로브 연결                   | 21 |

|          |                               |           |
|----------|-------------------------------|-----------|
| 3.6      | 사용자 공간 에이전트의 데이터 집계 . . . . . | 23        |
| <b>4</b> | <b>강화학습 기반 제어 에이전트 설계</b>     | <b>25</b> |
| 4.1      | 강화학습 문제 정형화 . . . . .         | 26        |
| 4.2      | 안전한 학습 및 적용 프레임워크 . . . . .   | 32        |
| 4.3      | eda_rl 실험 파라미터 표 . . . . .    | 33        |
| <b>5</b> | <b>실험 및 결과 분석</b>             | <b>35</b> |
| 5.1      | 실험 환경 설정 . . . . .            | 35        |
| 5.2      | 학습 수렴 분석 . . . . .            | 37        |
| 5.3      | 주요 시나리오별 성능 비교 분석 . . . . .   | 38        |
| 5.4      | 결과에 대한 고찰 . . . . .           | 42        |
| <b>6</b> | <b>결론 및 향후 연구</b>             | <b>42</b> |
|          | <b>References</b>             | <b>45</b> |

# 강화학습을 이용한 eBPF 커널 신호 기반 MQTT 꼬리 지연 제어에 관한 연구

**Jinuk Jung**

Department of Computer Science and Engineering  
Graduate School, Kyungpook National University  
Daegu, Korea

(Supervised by Professor Young-woo Kwon)

(Abstract) 사물인터넷(IoT) 확산으로 다수의 장치가 동시에 메시지를 발행하는 환경에서, 일부 요청의 지연이 수 초 이상으로 치솟는 꼬리 지연(tail latency) 문제가 빈번히 발생한다. 경량 발행/구독 프로토콜인 MQTT는 엣지 환경의 표준이지만, 네트워크 혼잡과 브로커 OS 큐잉으로 인한 극단 지연에 취약하다. 특히 혼잡이 누적될 경우 RTT 상승, 재전송 증가, 소켓 버퍼 포화가 연쇄적으로 발생하여 p99 지연이 급격히 악화된다. 기존 완화 기법은 클라이언트/브로커 수정이나 프로토콜 교체를 요구해 통합 비용과 운영 복잡도가 높다.

본 논문은 애플리케이션과 브로커 수정을 요구하지 않고 엣지 게이트웨이에 배포 가능한 지능형 제어 시스템 eMQTT-RL을 제안한다. eMQTT-RL은 리눅스 커널의 eBPF(extended Berkeley Packet Filter)로 RTT, 재전송, 송/수신 버퍼 점유율 등 커널 신호를 실시간 관측하고, 이를 상태로 사용해 강화학습(DRL, PPO)으로 MQTT 발행자의 전송률과 배치 크기를 동적으로 조절한다. 제어 문제를 마르코프 결정과정(MDP)으로 정형화하여, (1) 상태: 커널 신호 + 정책 맥락(현재 전송률/배치, 직전 행동), (2) 행동: 전송률 증감 비율과 배치 크기 단일 스텝 변화, (3) 보상: 처리량 유지 가중 + p99 지연 페널티로 설계하였다. 또한 보호 메커니즘을 도입해 스텝 크기 제한, 쿨다운, 혼잡 시 가속 억제, 처리량 바닥선 보장을 적용하여 실시간 환경에서의 안정성을 확보한다. 구현은 BCC 기반 eBPF 프로그램과 Python 에이전트로 구성되며, 배포가 용이하고 오버헤드는 수 퍼센트 수준으로 유지된다. VirtualBox 기반 테스트베드(Ubuntu 24.04, Linux 6.8, 4 vCPU/8 GB)에서 **EMQX/Paho** 워크로드와 netem 혼잡 시나리오로 성능을 평가하였다. 제안 기법은 무제어(Baseline) 및 MQTT-AC, 헤징 기법 대비 꼬리 지연을 크게 낮추었다. 애플리케이션 p99 지연(중앙값)은 무제어 55.9 s에서 제안 기법 0.48 s로 감소하였고, 커널 RTT p99는 1.89 s에서 0.78 s로 낮아졌다. 처리량은 171.7 msg/s(무제어) 대비 158.9 msg/s(제안)로 유사 수준을 유지하였다. 커널 레벨 p50/p95, 앱 p95 등 다른 분위수 지표에서도 일관된 개선을 확인했으며, 헤징과 고정 처리량 정책은 일시적 성능 향상과 맞바꾼 극단 꼬리 악화를 유발하였다. 이는 eBPF 기반 선행 지표를 활용해 큐잉 폭증 이전에 전송률을 선제적으로 조절함으로써, 처리량 저하 없이 꼬리 지연을 실질적으로

억제할 수 있음을 보여준다.

본 연구의 기여는 다음과 같다: (i) 커널 관측성(eBPF) 과 DRL 제어의 결합으로 애플리케이션/브로커 비침투적 꼬리 지연 제어 프레임워크 제시, (ii) 보호 메커니즘이 결합된 실시간 학습과 적용 파이프라인 설계, (iii) 다양한 혼잡 시나리오에서의 실험으로 꼬리 지연 대 처리량의 균형점을 정량화하고, 무수정 배포 가능성을 입증한다. 본 접근은 엣지 MQTT 통신의 신뢰성과 실시간성을 강화하는 실용적 해법으로, 향후 멀티 에이전트 확장, 학습 고도화, 다른 경량 프로토콜로의 일반화가 확장 가능하다.



# 1 서론

## 1.1 연구 배경: IoT 엣지 컴퓨팅과 MQTT의 부상

최근 사물 인터넷(IoT)은 스마트 팩토리, 자율 주행, 스마트 시티 등 산업 전반에 걸쳐 혁신을 주도하는 핵심적인 기술로 자리잡았다. 이러한 시스템들은 수천에서 수만 개에 이르는 센서와 액추에이터 장치들이 생성하는 방대한 양의 원격 측정(telemetry) 데이터를 수집하고 전파하는 능력에 의존한다. 데이터 처리의 패러다임은 중앙 집중형 클라우드에서 데이터가 생성되는 위치와 가까운 곳에서 처리하는 엣지 컴퓨팅(Edge Computing)으로 전환되고 있다. 엣지 컴퓨팅은 데이터 전송 지연 시간과 네트워크 대역폭을 줄이고 데이터 프라이버시를 강화하는 등 다양한 이점을 제공한다<sup>1,2</sup>.

이러한 분산된 엣지 환경에서 장치 간의 효율적인 통신을 위해서는 경량 메시징 프로토콜이 필요하다. 그중에서도 MQTT(Message Queuing Telemetry Transport)는 발행/구독(Publish/Subscribe) 모델을 기반으로 한 간결한 프로토콜 구조와 낮은 오버헤드 덕분에, 컴퓨팅 성능과 전력이 제한적인 IoT 장치 환경에서 표준으로 채택되었다<sup>3,4,5</sup>. MQTT는 중앙의 브로커(Broker)를 통해 메시지를 중개하고 송신자와 수신자를 분리하여 시스템의 확장성과 유연성을 크게 향상시킨다. 엣지 게이트웨이는 로컬 네트워크와 클라우드를 연결하는 중요한 지점에 위치하며, MQTT 브로커의 역할을 수행하는 경우가 많다. 이로

인해 엣지 게이트웨이는 수많은 장치로부터 트래픽이 집중되는 병목 지점이 될 수 있다.

## 1.2 문제 정의: 실시간 IoT 시스템의 꼬리 지연 시간

현대의 IoT 애플리케이션, 특히 스마트 팩토리의 로봇 제어나 커넥티드 카의 안전 시스템과 같이 실시간성이 중요한 분야에서는 평균적인 응답 시간뿐만 아니라 최악의 경우에 대한 응답 시간 보장이 매우 중요하다. 시스템의 전체 성능은 가장 느린 구성 요소에 의해 결정되는 경우가 많기 때문이다. 여기서 '꼬리 지연 시간(Tail Latency)'이라는 문제가 부상한다. 꼬리 지연 시간이란, 전체 요청 중 상위 95분위(p95) 또는 상위 99분위(p99)에 해당하는 극단적인 지연 시간을 의미한다<sup>6</sup>. 평균 지연 시간이 양호하더라도, 일부 메시지의 지연 시간이 수 초 단위로 급증하게 되면 실시간 제어 루프의 안정성을 해치고 전체 시스템의 신뢰도를 심각하게 저하시킬 수 있다.

네트워크 환경이 복잡하거나 부하가 큰 트래픽 환경은 꼬리 지연 시간 문제를 더욱 악화시킨다. 수많은 장치가 동시에 높은 빈도로 메시지를 발행하면, 발행자 측의 커널 송신 버퍼(sk\_wmem\_queued), 네트워크 장비, 그리고 브로커 내부의 수신/토픽/세션 큐 등 시스템 경로 전반에 걸쳐 큐가 쌓이게 된다. 이러한 큐잉(queueing) 현상은 지연 시간을 증폭시키는 주된 원인이다. 그래서 엣지 환경에 적용 가능한 꼬리 지연 시간 해결책은 애플리케이션이나 브로커 수정 없이, 엣지 게이트웨이에서 적은 리소스로 동작하며 점진적으로 배포할 수 있어

야 한다.

### 1.3 기존 접근법과 본 연구의 출발점

MQTT 꼬리 지연 시간을 완화하기 위해 브로커 내부 스케줄링 개선<sup>7,8</sup>이나 QUIC과 같은 새로운 전송 프로토콜 도입<sup>9</sup> 등이 시도되었으나, 특정 구현에 종속적이거나 인프라 전반의 변경을 요구하는 한계가 있다.

이러한 문제들을 해결하기 위해, 본 연구의 기반이 되는 선행 연구 'eMQTT-AC'는 eBPF를 이용해 커널 신호를 관측하고, 이를 바탕으로 발행자의 전송률을 조절하는 휴리스틱 기반 적응형 제어 시스템을 제안한 바 있다. 이 시스템은 애플리케이션이나 브로커 수정 없이 엷지 게이트웨이에서 동작하며 꼬리 지연 시간을 효과적으로 줄이는 성과를 보였다. 하지만 eMQTT-AC는 사전에 정의된 고정된 임계값(예: RTT가 70ms를 초과하면 제어)에 의존한다. 이러한 정적 규칙 기반의 휴리스틱은 특정 네트워크 환경에서는 효과적일 수 있으나, 패킷 손실률, 지터, 배경 트래픽 등이 수시로 변하는 동적이고 예측 불가능한 실제 엷지 환경에서는 최적의 성능을 보장하기 어렵다.

### 1.4 연구 목표 및 제안 방법: 강화학습 기반의 자율 제어

본 논문의 주된 연구 목표는 기존 휴리스틱 제어의 한계를 극복하고, 고도로 동적인 엷지 환경에서도 MQTT 꼬리 지연 시간을 효과적으로 최소화하는 지능형 제어 시스템, eMQTT-RL을 설계, 구현 및 평가하

는 것이다.

이를 위해 본 연구는 다음과 같은 가설을 설정한다: 심층 강화 학습(DRL) 에이전트는 eBPF를 통해 실시간으로 커널 신호를 직접 관측하고, 애플리케이션 수준의 지연 시간으로부터 피드백(보상)을 받음으로써, 사전에 프로그래밍된 휴리스틱 정책보다 더 효과적이고 강건한 제어 전략을 자율적으로 발견할 수 있다.

이 가설을 검증하기 위해, MQTT 꼬리 지연 시간 제어 문제를 마르코프 결정 과정(Markov Decision Process, MDP)으로 수학적으로 정형화한다. eBPF가 수집한 커널 신호(RTT, 재전송, 버퍼 점유율)를 '상태(State)'로, MQTT 발행자의 전송률 및 배치 크기 조정을 '행동(Action)'으로, 그리고 종단 간 지연 시간과 처리량을 기반으로 계산된 값을 '보상(Reward)'으로 정의한다. 또한, 온라인 학습 및 적용 과정의 안정성을 보장하기 위해 사전 정의된 규칙 기반의 보호 메커니즘을 도입한다. DRL 에이전트는 환경과의 지속적인 상호작용을 통해 시행착오를 겪으며, 장기적으로 누적 보상을 최대화하는 최적의 제어 정책(Policy)을 학습하게 된다. 이는 정적 규칙에서 벗어나, 데이터 기반의 동적이고 상황에 맞는 최적의 의사결정을 내리는 패러다임으로의 전환을 의미한다.

## 1.5 본 논문의 기여

본 논문이 기여하는 바는 다음과 같이 요약할 수 있다.

- **eBPF와 강화학습을 융합한 실시간 제어 프레임워크 설계:** 커널 수준의 저-오버헤드 관측과 애플리케이션 수준의 목표(SLO)를 결합하여, MQTT 꼬리 지연을 제어하는 새로운 DRL 기반 프레임워크 eMQTT-RL을 설계하고 구현하였다.
- **안전성을 고려한 점진적 학습 파이프라인 제안:** 새도우 모드를 통한 데이터 수집, 행동 복제를 이용한 오프라인 사전학습, 그리고 안전 가드가 적용된 온라인 미세조정으로 이어지는 실용적이고 점진적인 학습 파이프라인을 제안하여, 실제 운영 환경에 DRL 에이전트를 안전하게 배포하고 적용할 수 있는 방법론을 제시하였다.
- **지연-처리량 트레이드오프에 대한 실험적 분석:** 다양한 네트워크 시나리오에서 제안하는 시스템의 성능을 종합적으로 평가하고, 제어 유무에 따른 지연 시간과 처리량 사이의 상충 관계(trade-off)를 정량적으로 분석하여, 제어 정책의 운영점을 선택하는 기준을 제시하였다.

## 1.6 논문의 구성

본 논문은 다음과 같이 구성된다. 제2장에서는 MQTT 프로토콜, eBPF 기술, 그리고 네트워크 제어 분야에서의 강화학습 적용 사례 등 관련 연구를 심도 있게 고찰한다. 제3장에서는 제안하는 시스템의 핵심 기반 기술인 eBPF를 이용한 실시간 커널 상태 관측 방법론을 상세히 기술한다. 제4장에서는 제안하는 eMQTT-RL 시스템의 전체

아키텍처, MDP 모델링, 안전 가드 및 학습 절차를 상세히 기술한다. 제5장에서는 실험 환경을 구축하고, 제안하는 eMQTT-RL 시스템을 기저 상태 및 eMQTT-AC와 비교하여 성능을 정량적으로 평가하고, 학습된 정책을 분석한다. 마지막으로 제6장에서는 연구 결과를 요약하고, 본 연구의 의의와 한계, 그리고 향후 연구 방향을 제시하며 결론을 맺는다

## 2 관련연구

### 2.1 MQTT 프로토콜과 IoT 지연 문제

MQTT(Message Queuing Telemetry Transport)는 IoT 환경에서 널리 사용되는 경량 퍼블리시/구독(pub-sub) 기반 프로토콜이다<sup>3</sup>. MQTT는 중앙 브로커를 통해 메시지를 중계하며, 센서 노드 등의 퍼블리셔가 특정 토픽(topic)에 메시지를 발행하면, 해당 토픽을 구독한 구독자(subscriber)들에게 브로커가 메시지를 전달한다. MQTT는 TCP 위에서 동작하며 QoS(Quality of Service) 수준(0, 1, 2)별로 전달 신뢰도를 조절할 수 있는데, 높은 QoS의 경우 다중 확인 메시지로 오버헤드와 지연이 증가하는 상충 관계가 있다<sup>4</sup>. IoT 시스템은 대역폭이 제한적이고 기기 자원이 한정되므로, MQTT 통신에서 네트워크 혼잡, 브로커 부하, 메시지 크기 등에 따른 지연이 문제시된다<sup>7</sup>. 특히 브로커에 다수의 클라이언트가 접속하여 팬-아웃(fan-out) 규모가 큰 경우, 커널 네트워크 스택을 통한 잦은 컨텍스트 전환과 큐 대기로 엔드-투-엔드 지연이 수십 ms 이상 크게 증가할 수 있다. 또한 부하가 일정 임계치를 넘으면 지연이 일명 "하키-stick" 형태로 증가하며 시스템 처리량도 저하되는 현상이 관찰된다. 이러한 꼬리 지연은 IoT 제어 응용의 실시간성을 떨어뜨리고, 데이터 손실이나 응답 실패로 이어질 수 있어 반드시 해결이 필요하다. 지연의 주요 원인을 살펴보면, 네트워크 전송 지연(거리, 라우터 홉수), 큐잉 지연(브로커/네트워크 대기열 적체), 프로토콜 처리 지연(MQTT 패킷 파싱/직렬화) 등이 복합적으로

작용한다. IoT 기기 특성상 에너지 절약을 위한 슬립 모드나 활성화 지연이 존재하여, 요청 도착 시점과 실제 처리 가능 시점 사이에 구조적인 지연이 발생할 수 있다. 예를 들어 센서 노드가 저전력 모드에서 깨어나는 시간, 이전 주기의 작업 완료 대기 등이 누적되어 프로토콜 초기화 지연을 야기하며, 이는 일반적인 대기열 지연 모델에서 간과되기 쉽다. 결과적으로 버퍼 오버플로우, 처리 지연 증가, 패킷 손실 등이 발생하고, 심한 경우 임계 이벤트 누락이나 시스템 안전성 저해로 이어진다. 한편, MQTT-over-QUIC과 같은 최신 전송 연구는 TCP의 HOL 블로킹을 회피하고 보안/핸드셰이크 이점을 얻어 지연을 줄일 수 있음을 보였으나<sup>9</sup>, 고속 링크나 복잡한 인터넷 경로에서는 QUIC 자체도 최적이지 않을 수 있음이 보고되어 맥락 의존적이다<sup>10,11</sup>.

이러한 문제를 해결하기 위한 다양한 연구가 제안되었다. Arifin 등은 Round Robin과 Least Response Time 기반 브로커 스케줄링 기법을 비교 분석하여, 상황별 지연과 처리량 trade-off를 제시하였다<sup>7</sup>. Shvaika 등의 TBMQ(2025)는 브로커 내부 아키텍처 개선으로 확장성과 안정성을 높였다<sup>8</sup>. Yeh 등은 스마트팩토리 시나리오에서 MQTT를 적용하여, 실제 산업 환경에서 MQTT 지연이 제어 품질에 미치는 영향을 분석하였다<sup>5</sup>. 전송 스택 관점에서 MQTT-over-QUIC 구현과 평가가 보고되었고<sup>9</sup>, QUIC 성능에 대한 최신 평가도 병행되어 있다<sup>10,11</sup>. 이러한 연구들은 브로커 내부나 응용, 전송 계층 최적화에 집중했지만, 본 연구는 커널 계층(eBPF)에서 신호를 직접 수집하고 제어하여 지연 문제를 해결한다는 점에서 차별성을 가진다.



## 2.2 꼬리 지연(tail latency)과 제어 필요성

꼬리 지연은 전체 요청 중 상위 몇 퍼센트의 응답 지연이 평균에 비해 매우 큰 현상을 가리키며, 일반적으로 p95, p99와 같은 상위 분위 지연으로 측정된다. 분산 시스템에서 꼬리 지연은 소수 요청의 지연이 전체 서비스 품질을 결정하기 때문에 중요하다. 꼬리 지연을 완화하려는 전통적 기법으로는 여분 자원 할당, 요청 중복 실행(hedging), 우선순위 제어 등이 있다. 예를 들어 구글 등에서는 동일 요청을 여러 서버에 보내 가장 빨리 응답 오는 결과를 선택하는 헤지 요청으로 tail latency를 줄이는 시도를 했다<sup>12</sup>. 그러나 이러한 방법은 추가 자원 소모나 네트워크 부하를 유발하며, MQTT와 같은 단일 브로커 시스템에는 적용하기 어렵다.

또 다른 접근은 큐 관리 정책을 개선하는 것이다. 네트워크 라우터 분야에서는 DropTail, RED, CoDel 등의 알고리즘이 오래전부터 지연을 줄이고 큐를 안정화하기 위해 사용되어 왔다. DropTail은 버퍼가 가득 차면 신규 패킷을 버리는 단순 전략이며, RED는 조기 랜덤 드롭으로 혼잡을 예방하고, CoDel은 큐잉 시간이 임계치를 넘은 패킷을 버려 지연을 통제한다. IoT 환경에서도 LoRaWAN, NB-IoT 등에서 이러한 정책이 임베디드 형태로 쓰이고 있지만, 장치 내부 상태(예: 슬립 모드로 인한 처리 불능 시간)를 고려하지 않고 고정된 규칙으로 동작하므로, 이질적인 트래픽 패턴에는 취약하다.

전송 프로토콜 차원에서도 tail latency 개선을 위한 TCP 변형들

이 제안되었다. CUBIC은 대역폭-지연 곱이 큰 환경에서 빠른 수렴을 보장하도록 설계되었고<sup>13</sup>, DCTCP는 ECN 신호를 활용해 데이터센터 환경에서 짧은 지연과 높은 처리량을 달성하였다<sup>14</sup>. BBR은 대역폭과 RTT 추정을 기반으로 새로운 혼잡 제어 방식을 제시하며 tail latency 개선 효과를 입증하였다<sup>15</sup>. Mansour 등은 IoT 환경을 대상으로 TCP 변형들을 비교해, 장치 제약과 망 특성을 고려한 지연 제어 필요성을 제시하였다<sup>16</sup>. 최근에는 WiFi 망 특성에 맞는 혼잡 제어 재검토가 제안되는 등<sup>17</sup>, 환경별 최적 제어의 중요성이 커지고 있다.

한편 Netflix는 적응형 동시 접속 제한(Adaptive Concurrency Limit)을 통해 인스턴스 당 허용 요청 수를 지연 기반으로 동적으로 조절하는 기법을 적용하였다. 이 방법은 최소 지연 대비 현재 지연의 비율(gradient)을 이용해 큐잉 발생 여부를 감지하고, 지연이 증가하면 AIMD 유사 알고리즘으로 허용 동시 요청수를 줄인다. 그 결과 과도한 부하에서 추가 트래픽을 거부하여 지연 상승을 막고 서비스 가용성을 높였다. 그러나 이러한 기법들은 휴리스틱 규칙이나 고정 파라미터에 의존하는 경우가 많아 IoT 환경의 급격한 변동성에는 한계가 있다. 따라서 보다 지능적이고 적응적인 제어 전략이 필요하다.

## 2.3 eBPF를 활용한 커널 신호 관측

eBPF는 리눅스 커널(version 4.4 이상)에 내장된 고성능 샌드박스 실행 환경으로, 사용자 정의 코드를 커널에 안전하게 주입하여 동적으로

커널 동작을 추적하고 변경할 수 있다. 원래는 패킷 필터링을 위한 기술로 시작되었으나, 현재는 네트워킹, 보안, 성능 모니터링 등 광범위한 영역에서 사용되고 있다<sup>18</sup>.

eBPF 프로그램은 커널의 tracepoint, kprobe, socket, traffic control 등 다양한 지점에 연결되어 실행될 수 있으며, 커널 이벤트에 대한 콜백으로 동작하면서 실시간으로 데이터를 수집하거나 커널의 행동을 변경할 수 있다<sup>18</sup>. 예를 들어 eBPF를 이용하면 TCP 소켓의 send/ack 이벤트를 추적하여 RTT를 직접 측정하거나, TCP 재전송 발생 함수를 후크하여 재전송률을 계측할 수 있다<sup>19</sup>. Rezvani 등은 eBPF를 활용해 지연 민감 요청의 커널 내부 관측 가능성을 정량적으로 분석하여, 기존 사용자 공간 모니터링 대비 정밀성과 실시간성을 크게 향상시킬 수 있음을 보였다<sup>19</sup>. Liz Rice는 eBPF가 프로덕션 환경에서 성능 분석과 네트워크 제어를 위한 사실상 표준 도구로 자리잡았음을 강조하였다<sup>18</sup>.

특히 eBPF는 추가 하드웨어 장치 없이도 프로덕션 시스템에서 저오버헤드로 세부 성능 모니터링이 가능하다는 장점 때문에 Netflix, Facebook 등 대규모 서비스에서도 활용된다. Netflix는 eBPF 기반 스케줄러 모니터링으로 Noisy Neighbor 문제를 탐지하였고, Facebook은 커널 레이턴시 관측 툴을 개발하였다. 또한 eBPF를 통해 사용자 공간을 우회하는 네트워크 fast-path를 구현하면 RPC 프레임워크의 응답 지연을 줄일 수 있음이 보고되었다. 본 연구의 기반이 되는 eMQTT-AC에서도 eBPF를 통해 MQTT 통신의 RTT, 버퍼 점유율 등의 신호를 추출하고 메시지 흐름을 제어하였다.

## 2.4 강화학습을 이용한 네트워크 제어 동향

강화학습(RL)은 에이전트가 환경과 상호작용하며 시행착오를 통해 최적의 행동 정책을 학습하는 기계학습 기법이다. 최근 딥러닝 기술과 결합된 심층 강화학습(Deep RL)이 네트워크 제어 분야에도 도입되어 주목받고 있다. 복잡한 네트워크 환경에서는 전통적인 모델 기반 제어가 어려운 반면, RL은 환경으로부터 직접 피드백을 받아가며 비선형 최적 제어 전략을 스스로 학습할 수 있다는 장점이 있다.

심층 강화학습을 TCP 혼잡제어에 적용하여 기존 알고리즘 대비 지연과 처리량 모두에서 성능 개선을 한 연구와<sup>20</sup> DRL 기반 스케줄러 파라미터 튜닝을 통해 엣지 클라우드 작업의 tail latency를 줄일 수 있음을 보였다<sup>21</sup>. 최근 무선 네트워크에서도 RL 기반 혼잡 제어가 고정 파라미터 방식보다 효과적임이 보고되었다<sup>17</sup>.

이러한 선행연구들은 RL이 네트워크의 불확실성과 동적 변동성을 처리하는 데 특히 적합함을 보여준다. 따라서 IoT 메시징 시스템의 꼬리 지연 제어에도 RL을 도입하면, 사람이 일일이 파라미터를 조정하지 않아도 환경에 최적화된 제어 정책을 자동으로 학습할 수 있다는 가능성을 제공한다.

마지막으로, 본 논문과 직접 관련된 꼬리 지연 완화 전략들을 요약하여 비교하면 표 1와 같다. 클라이언트 측 헤징/백오프나 브로커 내부 최적화, 전송 스택 변경(예: MQTT-over-QUIC)과 달리, 본 연구의 eMQTT-RL은 애플리케이션/브로커 코드를 수정하지 않고 엣지에서

퍼블리셔 전송률/배치를 제어함으로써 배포 용이성과 효과를 동시에 추구한다. 전송률은 토큰 버킷 기반 페이싱으로, 배치는 크기+플러시 지연 정책으로 구현되며(자세한 구현은 4장 ??), 커널 신호를 활용한 학습형 제어로 변동성이 큰 IoT 환경에서의 일반화 가능성을 높인다.

**Table 1: 테일 지연 완화 전략 비교**

| 전략                 | 변경 범위                      | 조절 대상                      | 배포 난이도             |
|--------------------|----------------------------|----------------------------|--------------------|
| 클라이언트<br>헤징/백오프    | 클라이언트                      | 중복요청/재시<br>도/백오프           | 어려움 (대규모 단말)       |
| 브로커 내부 최적화         | 브로커                        | 큐/스케줄링/백프레셔                | 중간 (벤더 의존)         |
| MQTT-over-QUIC     | 클라이언트, 브로커                 | 전송 스택 (QUIC 매핑)            | 어려움 (호환성)          |
| DRL 기반 전송 제어       | 커널/전송계층,<br>학습/추론 파이프라인    | TCP 파라미터 (cwnd,<br>pacing) | 어려움 (데이터/학습<br>필요) |
| eMQTT-RL (본<br>연구) | 애플/브로커 무변경,<br>학습/관측 파이프라인 | 퍼블리셔 전송률/배치<br>(커널 신호 기반)  | 중간 (학습/모니터링<br>필요) |

### 3 eBPF 기반 실시간 커널 관측

본 장에서는 제안하는 제어 시스템 eMQTT-RL의 핵심 기반 기술인 eBPF를 이용한 실시간 커널 상태 관측 방법론을 상세히 기술한다. 애플리케이션 수준의 지표만으로는 파악하기 어려운 네트워크의 미시적 혼잡 상태를 감지하기 위해, 본 연구는 커널 TCP/IP 스택에서 직접 주요 성능 신호를 저-오버헤드로 추출한다. 이는 에이전트가 꼬리 지연 발생을 사전에 예측하고 선제적으로 대응하는 데 필수적인 선행 지표(leading indicator)를 제공한다.

### 3.1 꼬리 지연 제어의 중요성

본 연구의 목표는 단순한 평균 지연이 아니라, 서비스 품질을 결정짓는 상위 분위 지연(p99 latency)을 통제하는 데 있다. 실시간 엣지 워크로드에서 서비스 수준 협약(SLA) 위반은 주로 p95나 p99 구간에서 발생하며, 이는 전체 시스템의 응답성과 안정성에 연쇄적인 영향을 미친다.

꼬리 지연을 효과적으로 제어하는 것은 확률적 SLA를 충족하기 위한 핵심 요인이다. 시간 민감형 워크로드의 SLA는 “99%의 요청이  $D$  ms 이내에 처리되어야 한다”와 같은 형태로 정의되는 경우가 많으며, p99 지연을 직접적으로 관리함으로써  $P(\text{latency} > D)$  확률을 낮출 수 있다. 이는 시스템의 예측 가능성과 신뢰성을 높여 제어 지연으로 인한 안전사고나 생산성 저하의 가능성을 감소시킨다. 또한 지연이 불안정한 환경에서는 클라이언트의 타임아웃 및 재시도로 인해 네트워크 부하가 가중되는 “retry storm” 현상이 발생할 수 있다. 이러한 반복 요청은 컨텍스트 스위치와 인터럽트 처리 비용을 증가시켜 처리율을 저하시킨다. 반면, 꼬리 지연을 안정화하면 재전송 및 타임아웃 빈도가 줄어들어 시스템의 효율성과 처리 안정성이 향상된다. 그리고 안정적인 지연 시간은 자원 관리와 비용 측면에서도 중요한 이점을 제공한다. 브로커의 불필요한 스케일아웃이나 네트워크 링크 증설을 줄일 수 있으며, 메시지가 큐에 장시간 머물며 신선도를 잃는 비율도 감소한다. 이는 결과적으로 엣지 장치의 CPU와 배터리 소모를 줄여 전반적인 에너지 효율을 높이는 효과를 가진다. 그리고 꼬리 지연의

완화는 지연 분산(jitter)의 감소로 이어져 제어 루프의 안정성을 강화한다. 지연의 변동성이 줄어들면 제어기의 파라미터를 보다 적극적으로 조정하더라도 시스템의 안정성이 유지되며, 이는 제어 성능의 향상으로 직결된다.

이와 같이, 시스템의 신뢰성과 가용성은 평균 지연보다는 최악의 지연에 의해 좌우된다. 따라서 꼬리 지연을 억제하는 것은 실시간 엣지 환경에서 안정적인 서비스를 보장하기 위한 핵심 전략이라 할 수 있다.

### 3.2 eBPF 커널 관측의 당위성

꼬리 지연을 제어하기 위해서는 제어 에이전트가 시스템의 상태를 빠르고 정확하게 인지할 수 있어야 한다. 본 연구는 이러한 요구를 충족하기 위한 방안으로 eBPF를 활용한 커널 수준 관측 방식을 채택하였다. 이는 기존 애플리케이션 계층의 지표만으로는 실시간 제어에 필요한 반응성을 확보하기 어렵기 때문이다. 먼저, eBPF 기반 관측은 문제를 사전에 탐지할 수 있는 선제적 신호를 제공한다. 애플리케이션 레벨에서 측정되는 p99 지연은 보통 수십 초 단위의 윈도우 집계 결과로, 이미 성능 저하가 발생한 이후에야 확인 가능한 사후 지표에 가깝다. 반면 eBPF를 통해 수집되는 RTT, 재전송 이벤트, 소켓 버퍼 점유율 등의 커널 신호는 밀리초에서 수 초 단위의 즉각적인 변화를

반영하므로, 제어 에이전트가 혼잡의 징후를 조기에 감지하고 선제적으로 대응할 수 있다. 또한, eBPF는 커널 레벨에서 특정 포트나 소켓 이벤트를 후킹(hooking)하는 방식으로 동작하므로, 기존 퍼블리셔나 브로커 애플리케이션의 코드를 수정할 필요가 없다. 운영 중인 시스템에 비침투적으로 관측 및 제어 로직을 삽입할 수 있어, 산업 환경에서도 높은 적용성을 확보할 수 있다.

커널을 직접 관측하는 행위는 병목 지점의 정확한 식별에도 유리하다. 꼬리 지연은 TCP 혼잡 윈도우 축소, 네트워크 인터페이스 큐 적체, 재전송 누적 등 다양한 내부 지점에서 발생하지만, 애플리케이션 계층에서는 단순히 “응답이 느리다”는 현상만 관찰될 뿐 원인을 구분하기 어렵다. 반면 커널 계층의 관측은 각 지점의 상태를 세분화된 신호로 제공하므로, 강화학습 에이전트가 보다 정확한 상황 인식과의 사결정을 수행할 수 있다. 그리고 eBPF는 낮은 오버헤드와 높은 관측 정밀도 또한 보장한다. 커널 내부에서 직접 실행되기 때문에 사용자 공간으로의 데이터 복사 비용이 거의 없으며, 수 퍼센트 미만의 부하로 실시간 동작이 가능하다. 더불어, 특정 브로커 포트나 토픽 단위의 흐름(per-flow)에 대한 세밀한 관측이 가능하여, 다수의 트래픽이 공존하는 환경에서도 개별 흐름의 상태를 분리하여 분석할 수 있다.

결국 eBPF는 강화학습 기반 제어에 필요한 핵심 상태 정보를 기존 방식보다 빠르고 정확하게 제공한다. 애플리케이션 지표만으로는 이미 문제가 발생한 뒤 대응하는 데 그치지만, 커널 신호는 지연이 폭발적으로 증가하기 이전 단계에서 제어 결정을 내릴 수 있는 근거를 제시한다. 특히 대기행렬 이론에 따르면 시스템의 혼잡도  $\rho = \lambda/\mu$ 가



1에 근접할수록 평균 대기시간은 급격히 증가한다( $W_q \approx \rho/(\mu - \lambda)$ ). 이는 큐의 압력이 상승하는 초기에 제어가 이루어져야 함을 시사하며, eBPF 기반 커널 관측이 실시간 제어의 필수 요소임을 뒷받침한다.

**Table 2:** eBPF 커널 관측 신호 요약

| 신호                                        | 의미        | 중요성                    |
|-------------------------------------------|-----------|------------------------|
| RTT (srtt_us)                             | 경로 지연     | 혼잡/경로 변동에 민감한 선행 지표    |
| 재전송 증가량 ( $\Delta$ of tcp_retransmit_skb) | 손실/혼잡 신호  | 버스트/손실 감지, 꼬리 지연 직결    |
| 송신 버퍼압 ( $sk\_wmem\_queued/sndbuf\_max$ ) | 퍼블리셔 측 큐잉 | sender 병목/큐 체류 증가      |
| 수신 버퍼압 ( $sk\_rmem\_alloc/rcvbuf\_max$ )  | 브로커 측 큐잉  | receiver/브로커 병목, 지연 증가 |

### 3.3 eBPF 기술 개요

eBPF는 리눅스 커널 내에서 샌드박스 형태로 사용자 정의 코드를 안전하게 실행할 수 있게 하는 혁신적인 기술이다<sup>18</sup>. 원래 패킷 필터링을 위해 설계되었으나, 기능이 크게 확장되어 현재는 네트워킹, 보안, 성능 모니터링 등 광범위한 영역에서 커널의 동작을 동적으로 추적하고 변경하는 데 사용된다. eBPF는 커널 코드의 수정이나 재컴파일 없이, kprobe, tracepoint, XDP 등 다양한 훅(hook)을 통해 특정 이벤트 발생 시점에 프로그램을 주입할 수 있다. 주입된 코드는 실행 전 검증기(verifier)에 의해 안정성을 검증받으므로, 커널 패닉 등의 위험 없이 안전하게 실행된다. 본 연구에서는 이러한 eBPF의 특성을 활용하여, MQTT 통신과 관련된 TCP 세션의 내부 상태를 실시간으로 관측한다.

### 3.4 핵심 관측 신호

꼬리 지연의 근본적인 원인인 네트워크 혼잡과 큐잉 지연을 효과적으로 감지하기 위해, 다음과 같은 세 가지 핵심 커널 신호를 관측 대상으로 선정하였다. 이들은 꼬리 지연 발생의 직접적인 원인이 되거나, 발생 직전에 급격한 변화를 보이는 선행 지표이다.

- **왕복 시간(Round-Trip Time, RTT):** 패킷이 목적지까지 도달하고 확인 응답(ACK)이 돌아오는 데 걸리는 시간이다. RTT의 증가는 네트워크 경로상의 큐잉 지연 증가를 의미하며, 혼잡의 가장 직접적인 신호이다. 본 연구에서는 TCP 소켓 정보에 저장된 평활화된 RTT(`srtt_us`) 값을 추적한다. 이는 TCP 혼잡 제어 알고리즘(예: BBR, CUBIC)에서도 핵심적으로 사용되는 지표이다<sup>13,15</sup>.
- **TCP 재전송(TCP Retransmission):** 패킷 손실이 발생했음을 의미하며, 네트워크 혼잡의 명백한 증상이다. 재전송이 발생하면 RTO(Retransmission Timeout) 만큼의 지연이 추가되어 꼬리 지연에 직접적인 영향을 미친다. 본 연구에서는 `tcp_retransmit_skb` 트레이스포인트를 이용해 재전송 이벤트 발생을 직접 계측한다.
- **소켓 버퍼 점유량(Socket Buffer Occupancy):** 송신(send) 및 수신(receive) 소켓 버퍼에 쌓여있는 데이터의 양이다. 송신 버퍼 점유량(`sk_wmem_queued`)의 증가는 애플리케이션이 생성

하는 데이터 속도를 네트워크가 처리하는 속도보다 빠르다는 것을 의미하며, 이는 곧 큐잉 지연으로 이어진다. 수신 버퍼 점유량(sk\_rmem\_alloc)의 증가는 수신 측 애플리케이션이 데이터를 충분히 빨리 소비하지 못하고 있음을 나타낸다. 이 두 지표는 애플리케이션이 인지하기 전에 커널 수준에서 발생하는 병목 현상을 포착하게 해준다<sup>19</sup>.

### 3.5 eBPF 프로그램 구현

상기 신호들을 수집하기 위해 BCC(BPF Compiler Collection) 프레임워크를 사용하여 C로 eBPF 프로그램을 작성하고, 이를 사용자 공간의 Python 제어 에이전트에서 불러와 관리한다. 프로그램의 핵심 로직은 다음과 같다.

#### 3.5.1 데이터 구조

커널 공간에서 수집된 정보를 사용자 공간으로 전달하기 위해 BPF\_HASH 타입의 맵(map)을 사용한다. 이 해시 맵(stats)은 4-튜플(송신/수신 IP, 포트)로 구성된 TCP 흐름(flow)을 키(key)로, 해당 흐름의 RTT, 재전송 횟수, 버퍼 점유량 등을 값(value)으로 저장한다.

### 3.5.2 커널 프로브 연결

특정 커널 이벤트 발생 시점에 우리가 작성한 코드를 실행시키기 위해 kprobe와 tracepoint를 사용한다.

- `tcp_rcv_established`: TCP 연결이 설정된 상태에서 패킷을 수신할 때마다 호출되는 함수다. 여기에 kprobe를 연결하여, 해당 TCP 소켓의 `struct tcp_sock` 구조체에 접근하고, 이 구조체에 저장된 `srtt_us`(마이크로초 단위의 Smoothed RTT) 값과 `sk_rmem_alloc`(수신 버퍼 사용량) 값을 읽어와 BPF 맵에 갱신한다.
- `tcp_sendmsg`: 애플리케이션이 TCP 소켓을 통해 데이터를 전송할 때 호출되는 함수다. 여기에 kprobe를 연결하여, 현재 송신 버퍼에 쌓인 데이터의 양(`sk_wmem_queued`)을 읽어 BPF 맵에 갱신한다.
- `tcp_retransmit_skb` tracepoint: TCP 스택이 패킷 재전송을 시도할 때마다 활성화된다. 여기에 연결된 eBPF 프로그램은 재전송 카운터를 1 증가시켜 BPF 맵에 기록한다.

아래는 RTT와 수신 버퍼를 수집하는 eBPF 코드의 일부이다.

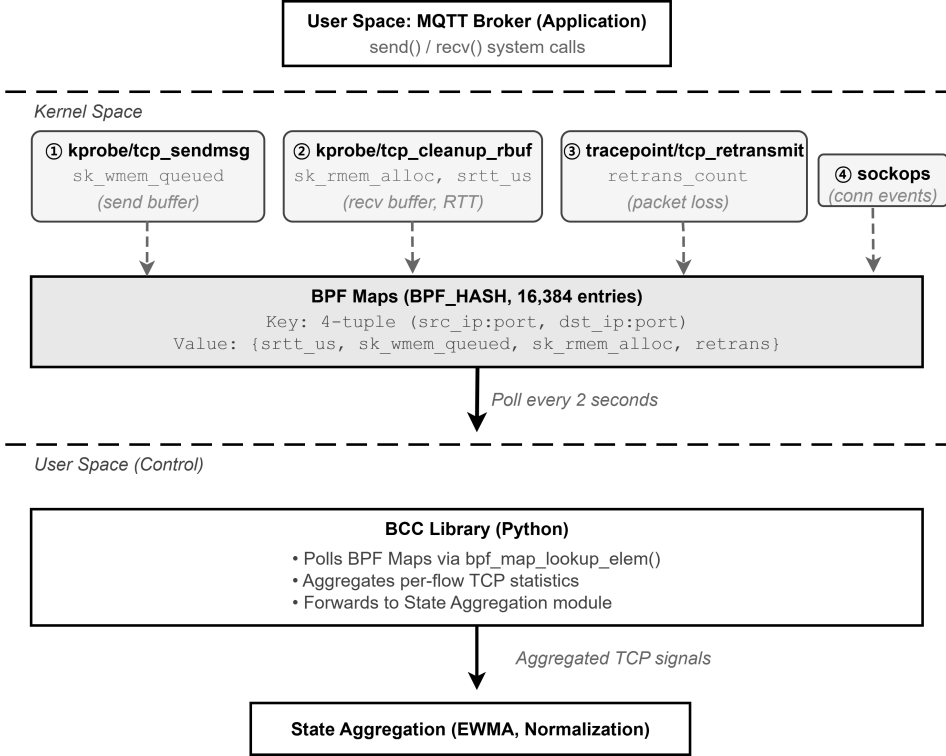
```
1 int on_tcp_rcv_established(struct pt_regs *ctx, struct sock *sk) {
2     if (!sk) return 0;
3     struct key_t k = {};
```

```

4  if (!fill_key(sk, &k)) return 0; // Tracked port filtering
5
6  struct tcp_sock *tp = (struct tcp_sock *)sk;
7  __u32 srtt = 0;
8  bpf_probe_read_kernel(&srtt, sizeof(srtt), &tp->srtt_us);
9  if (srtt) srtt >>= 3; // srtt is stored as 8x the actual value
10
11  __u32 rmem = 0;
12  bpf_probe_read_kernel(&rmem, sizeof(rmem), &sk->sk_rmem_alloc);
13
14  struct val_t zero = {};
15  struct val_t *v = stats.lookup_or_init(&k, &zero);
16  if (v) {
17      v->srtt_us = srtt;
18      v->rcvbuf = rmem;
19  }
20  return 0;
21 }

```

그림 1은 본 연구의 커널 신호 수집 파이프라인을 요약한다. kprobe/tracepoint로 수집된 TCP 신호가 BPF 해시 맵에 축적되고, 사용자 공간에서는 BCC를 통해 2 주기로 맵을 폴링하여 흐름별 통계를 읽어온 뒤 상태 집계를 수행한다.



**Fig. 1:** Kernel telemetry pipeline with eBPF. TCP hooks

### 3.6 사용자 공간 에이전트의 데이터 집계

사용자 공간에서 실행되는 Python 제어 에이전트는 BCC 라이브러리를 통해 eBPF 맵에 저장된 커널 통계를 주기적으로 읽어 TCP 플로우별 데이터를 집계한다. 에이전트는  $T_{\text{poll}} = 1$ 초 주기로 stats 맵을 폴링하여 각 플로우(4-tuple)의 RTT, 버퍼 점유량, 재전송 이벤트를 수집한다. 단일 주기 내에 여러 TCP 흐름이 관측될 수 있으므로, 에이전트는 이 값들을 집계하여 시스템 전체의 상태를 나타내는 단일 값으로 변환한다. RTT는 모든 활성 흐름의 RTT 값에 대해 지수가중 이동평균(EWMA)을 적용하여 현재의 평균적인 네트워크 지연 상태

를 추정한다. EWMA는 가중 계수  $\alpha = 0.3$ 을 사용하며, 다음과 같이 계산된다:

$$\text{RTT}_{\text{smooth}}(t) = \alpha \cdot \text{RTT}_{\text{raw}}(t) + (1 - \alpha) \cdot \text{RTT}_{\text{smooth}}(t - 1)$$

이를 통해 순간적인 RTT 스파이크를 완화하고 안정적인 신호를 얻는다. 버퍼 점유량은 관측된 최대값을 사용하며, 송신 버퍼는  $\text{sk\_wmem\_queued} / \text{sndbuf\_max}$ , 수신 버퍼는  $\text{sk\_rmem\_alloc} / \text{rcvbuf\_max}$ 로 정규화된다. 재전송은 발생 여부를 나타내는 이진 플래그로 집계된다. 모든 연속형 신호는 최대값을 기준으로 한 클리핑-정규화(min-max with cap)를 적용하여  $[0, 1]$  범위로 변환한다. 기본 정규화는  $x_{\text{norm}} = \min(1, x/x_{\text{max}})$ 로 정의하며, RTT는  $x_{\text{RTT}}^{\text{max}} = 300 \text{ ms}$ , 재전송율은  $x_{\text{ret}}^{\text{max}} = 10/\text{s}$ 를 기준으로 한다. 비활성 플로우(60초 이상 업데이트 없음)는 메모리 관리를 위해 자동으로 제거된다. 이렇게 집계되고 정규화된 값들이 다음 장에서 설명할 강화학습 에이전트의 상태 벡터(state vector)  $\mathbf{s}_t \in \mathbb{R}^9$ 로 사용된다. 상태 벡터는 커널 신호 다섯 가지(RTT, 송신/수신 버퍼 점유율, 재전송, 종합 큐 압력)와 정책 맥락 세 가지(현재 전송률, 배치 크기, 직전 행동)를 포함하며, 이는 4.1절에서 상세히 기술한다.

## 4 강화학습 기반 제어 에이전트 설계

본 장에서는 3장에서 설명한 eBPF 기반 커널 관측 시스템을 활용하여 MQTT 트래픽을 지능적으로 제어하는 강화학습 에이전트, eMQTT-RL의 설계 방법론을 상세히 기술한다. 그림 2는 제안하는 eMQTT-RL 시스템의 전체 구조를 보여준다. 시스템은 크게 3개 계층으로 구성되며, 각 계층의 역할은 다음과 같다.

첫째, 디바이스 계층에서는 20개의 MQTT Publisher가 각각 50 msg/s로 전송하여 총 1,000 msg/s 부하를 형성하고 control/room1 토픽을 구독하여 속도 제어 명령을 수신한다. Subscriber는 종단간 지연시간을 측정하여 e2e-latency/room1 토픽으로 보고한다.

둘째, 커널 공간에서는 eBPF Agent가 4개의 커널 훅(kprobe/tcp\_sendmsg, kprobe/tcp\_cleanup\_rbuf, tracepoint/tcp\_retransmit\_skb, sockops)을 통해 RTT, 송수신 버퍼, 재전송 등의 TCP 신호를 실시간으로 수집하여 16,384개 엔트리를 가진 BPF Maps(BPF\_HASH)에 저장한다.

셋째, 사용자 공간 제어 파이프라인에서는 BCC Library가 2초마다 BPF Maps를 폴링하여 수집된 TCP 통계를 읽어온다. State Aggregation 모듈은 EWMA 스무딩( $\alpha = 0.3$ )과 정규화를 수행하여 9개 특성(커널 신호 5개, 제어 인지 2개, 모멘텀 2개)으로 구성된 상태 벡터를 생성한다. RL Agent는 Rule 기반 또는 PPO 알고리즘을



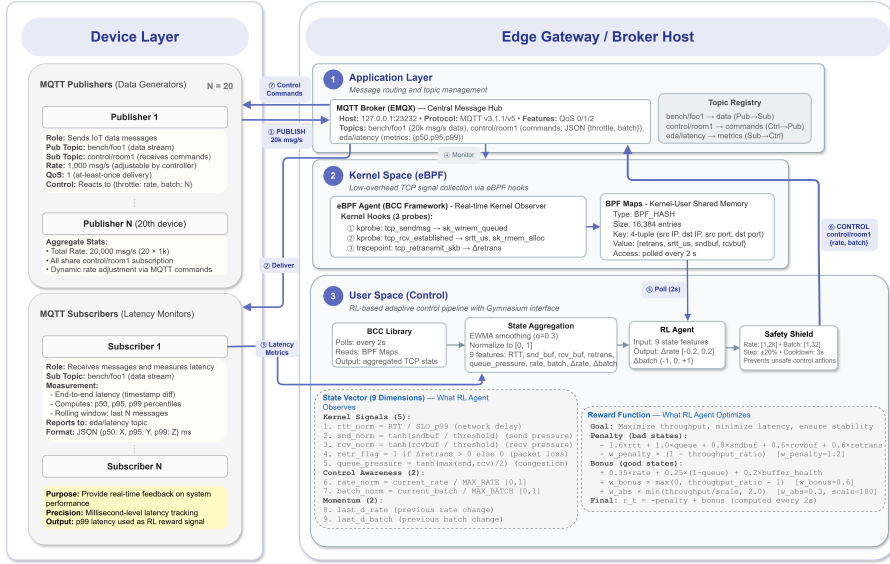
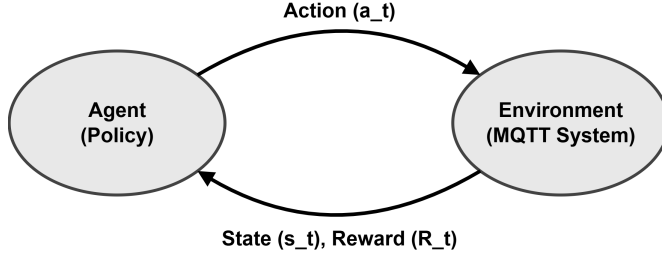


Fig. 2: eMQTT-RL 시스템 아키텍처

사용하여 속도 변화량( $\Delta rate$ )과 배치 변화량( $\Delta batch$ )을 결정하고, Safety Shield가 물리적 한계(rate:  $[1, 2,000]$ , batch:  $[1, 32]$ , step:  $\pm 20\%$ , cooldown: 3초)와 안전성을 검증한 후 최종 제어 명령을 MQTT Broker를 통해 Publisher들에게 전달한다.

## 4.1 강화학습 문제 정형화

본 제어 문제를 시간 이산 마르코프 결정 과정(Time-discrete Markov Decision Process, MDP)으로 정형화한다. MDP는  $(S, A, P, R, \gamma)$ 의 튜플로 정의되며, 목표는 할인된 누적 보상의 기댓값을 최대화하는 최적 정책  $\pi^*$ 를 찾는 것이다.



**Fig. 3:** MDP에서의 에이전트-환경 상호작용

그림 3은 Agent와 Environment의 순환 구조를 보여준다. Agent (정책  $\pi$ )는 Environment(MQTT System)로부터 상태  $s_t$ 와 보상  $R_t$ 를 관찰하고, 정책에 따라 행동  $a_t$ 를 선택한다. Environment는 행동을 실행하여 다음 상태  $s_{t+1}$ 로 전이하며, 이 과정이 반복된다. 본 연구의 목표는 할인된 누적 보상의 기댓값을 최대화하는 최적 정책  $\pi^*$ 를 찾는 것이다.

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[ \sum_{t=0}^T \gamma^t R_t \mid \pi \right] \quad (1)$$

여기서  $\gamma \in [0, 1]$ 은 할인 계수(discount factor)로, 미래 보상의 현재 가치를 조정한다. 본 연구에서는  $\gamma = 0.99$ 를 사용한다. 에이전트가 시점  $t$ 에 관측하는 상태 벡터  $s_t \in \mathbb{R}^9$ 는 3장에서 설명한 커널 신호 다섯 가지, 정책 맥락 두 가지, 모멘텀 두 가지를 포함하는 9개의 정규화된 특성으로 구성된다. 커널 신호에는  $RTT(s_{rtt})$ , 송신 및 수신 버퍼 점유율( $s_{snd}, s_{rcv}$ ), 재전송 발생( $s_{ret}$ ), 종합 큐 압력( $s_q$ )이 포함되며, 정책 맥락에는 현재 전송률( $s_{rate}$ ), 배치 크기( $s_{batch}$ )가 포함되고, 모멘텀에는 직전 스텝의 전송률 변화량( $\Delta s_{rate}$ )과 배치 변화량( $\Delta s_{batch}$ )이 포함된

다. 모든 연속형 신호는 최대값을 기준으로 한 클리핑-정규화(min-max with cap)를 적용하고, 지터 억제를 위해 지수가중이동평균(EWMA, 창 200 ms)을 사용한다. 기본 정규화는  $x^{\text{norm}} = \min(1, x/x_{\max})$  로 정의한다. 본 연구에서 사용한 한계치는 다음과 같다: RTT는  $x_{\max}^{\text{RTT}} = 300 \text{ ms}$ 로 두고  $s_{rtt} = \min(1, \text{RTT}^{\text{p95}}/300 \text{ ms})$ 로 정규화하며, 송신 및 수신 버퍼는 각각  $x_{\max}^{\text{snd}} = 0.8 \cdot \text{sndbuf}$ ,  $x_{\max}^{\text{rcv}} = 0.8 \cdot \text{rcvbuf}$ 를 기준으로 비율 정규화를 수행하여  $s_{\text{snd}}, s_{\text{rcv}}$ 를 얻는다. 재전송율은  $x_{\max}^{\text{ret}} = 10/\text{s}$ 에서  $s_{\text{ret}} = \min(1, \text{ret\_rate}/10)$ 로 두고, 종합 큐 압력은

$$s_q = \text{clip}(0, 0.4 s_{rtt} + 0.4 \max(s_{\text{snd}}, s_{\text{rcv}}) + 0.2 s_{\text{ret}}, 1)$$

로 정의한다. 전송률과 배치는 각각  $s_{\text{rate}} = r_t/r_{\max}$ ,  $s_{\text{batch}} = b_t/b_{\max}$ 이며, 본 실험에서  $r_{\min}=100$ ,  $r_{\max}=300 \text{ msg/s}$ ,  $b_{\min}=1$ ,  $b_{\max}=8$ 을 사용한다. 에이전트의 행동  $A_t$ 는 MQTT 퍼블리셔의 트래픽 패턴을 제어하기 위한 두 가지의 복합적 파라미터로 구성된다. 첫째, 전송률 변화량  $\Delta r_t$ 는 현재 전송률  $r_t$ 에 대한 상대적 변화율로서  $\Delta r_t \in [-0.2, 0.2]$ 의 연속 공간을 갖고, 상황에 따라 미세하거나 과감한 비례 제어를 가능하게 한다. 둘째, 배치 크기 변화량  $\Delta b_t$ 는 현재 배치 크기  $b_t$ 를 한 단계 증감하거나 유지하는 이산 공간  $\{-1, 0, +1\}$ 을 갖고, 메시지 전송의 버스트(burstiness)를 조절하여 큐 체류 시간 분포—특히 꼬리 지연—에 직접 영향을 미친다. 이처럼 전송의 양적 측면(전송률)과 질적 측면(전송 패턴)을 분리하여 제어함으로써 다각적인 최적화를 수행할 수 있다.

에이전트가 산출한 전송률( $r_t$ )과 배치 크기( $b_t$ )는 퍼블리셔 측에서  
 경량 제어로 즉시 반영된다. 전송률은 토큰 버킷(rate limiter) 기반 페  
 이싱으로 집행되며, 시간당  $r_t$  토큰을 보충(refill)하고 메시지 1건 발행  
 시 토큰 1개를 소모한다. 토큰이 없으면 다음 보충 시점까지 대기하므  
 로, 구현상 목표 간격  $\Delta t = 1000 \text{ ms} / r_t$ 를 이용한 고해상도 페이싱과  
 동등하다. 다중 퍼블리셔가 존재하는 경우 합성률이  $r_t$ 를 초과하지 않  
 도록 중앙 제한기(limiter)가 일관되게 제어한다. 배치는 크기  $b_t$ 에 도  
 달하거나 최대 플러시 지연  $T_{\text{flush}}$ (예: 수 ms)가 경과하면 즉시 플러시  
 되며, 즉  $|\text{batch}| = b_t$  또는 타이머 만료 중 먼저 발생한 이벤트에 따라  
 누적된 메시지를 연속 전송한다. 새로운  $(r_t, b_t)$ 는 원자적으로 갱신하  
 며, 제어의 과도한 요동을 막기 위해 변화폭 제한과 쿨다운은 § 4.2의  
 보호 메커니즘을 따른다. 또한 RTT 상승 및 소켓 버퍼 점유율 증가 등  
 커널 혼잡 신호가 임계치를 넘으면, 안전 규칙에 의해  $b_t$ 를 일시적으로  
 축소(예: 1로 캡)하거나  $r_t$ 에 감쇠 계수를 곱해 선제적으로 페이싱을  
 강화한다. 아래는 구현 의사코드(간단화)이다.

```

1 # apply_action( $\Delta(r_t, \Delta b_t) \rightarrow (r_{\text{next}}, b_{\text{next}})$ )
2  $r_{\text{next}} = \text{clamp}(r_t * (1 + \Delta r_t), r_{\text{min}}=100, r_{\text{max}}=300)$ 
3  $b_{\text{next}} = \text{clamp}(b_t + \Delta b_t, b_{\text{min}}=1, b_{\text{max}}=8)$ 
4
5 # token-bucket pacing (central limiter)
6 while running:
7     wait_until(next_send_time)
8     if tokens > 0:
9         publish(batch.pop() or read_next())
10        tokens -= 1

```

```

11     next_send_time += 1.0 / r_next
12     else:
13         sleep(token_refill_interval)
14
15     # batching policy (size OR time)
16     buffer.append(msg)
17     if len(buffer) >= b_next or (now - last_flush) >= T_flush_max:
18         send(buffer); buffer.clear(); last_flush = now

```

보상 함수  $R_t$ 는 에이전트의 학습 목표를 정의하며, 꼬리 지연 최소화과 처리량 유지라는 상충될 수 있는 두 목표 사이의 균형을 맞추도록 설계하였다. p99 지연이 서비스 수준 목표(SLO)를 초과한 정도에 비례하여 강한 음수 보상을 부과하고, 목표 처리량을 달성하면 양의 보상을 제공하여 과도한 전송 억제를 방지한다. 또한 애플리케이션 p99 지표의 수신 지연을 보완하기 위해 RTT와 큐 압력 등 실시간 커널 신호를 대리 항으로 포함한다. 이상의 요소는 다음과 같은 가중 합 형태로 결합된다.

$$\begin{aligned}
R_t = & \underbrace{w_{\text{thr}} \cdot (\min(\text{thr}_t, \text{thr}^{\text{target}}) / \text{thr}^{\text{target}})}_{\text{처리량 정규화 보상}} \\
& - \underbrace{w_{\text{slo}} \cdot (\max(0, \text{p99}_t - \text{SLO}) / \text{SLO})}_{\text{p99 초과율 페널티}} \\
& - \underbrace{w_{\text{rtt}} \cdot (\text{RTT}_t^{\text{p99}} / \text{RTT}_{\text{ref}}^{\text{p99}})}_{\text{커널 RTT 상대 페널티}} \\
& - \underbrace{w_{\text{buf}} \cdot (\text{qpress}_t)}_{\text{큐 압력(0~1)}} \quad (2)
\end{aligned}$$

여기서  $\text{thr}_t$ 는 측정 처리량,  $\text{thr}^{\text{target}}$ 은 목표 처리량(예: 300 msg/s),  $\text{p99}_t$ 는 앱 레벨 p99 지연(ms),  $\text{RTT}_t^{\text{p99}}$ 는 커널 RTT p99,  $\text{RTT}_{\text{ref}}^{\text{p99}}$ 는 기준값(예: 200 ms),  $\text{qpress}_t \in [0, 1]$ 는 버퍼 점유와 재전송으로 구성된 정규화된 큐 압력이다. 기본 가중치는  $w_{\text{thr}}=1.0$ ,  $w_{\text{slo}}=2.0$ ,  $w_{\text{rtt}}=0.5$ ,  $w_{\text{buf}}=0.25$ 로 설정하고, 민감도 분석은 부록에서 제시할 수 있다. 아래는 구현 의사코드이다.

```

1 thr_term  = min(thr_t, thr_target) / thr_target
2 slo_excess = max(0, p99_t - SLO) / SLO
3 rtt_term   = rtt_p99_t / rtt_ref
4 reward    = 1.0*thr_term - 2.0*slo_excess - 0.5*rtt_term - 0.25*qpress_t

```

규칙기반 제어에서 빈번히 마주치는 관측 패턴과 권장 행동은 표 3에 요약하였다. 이는 5장에서의 베이스라인 비교 시 참조 기준으로 활용한다.

**Table 3:** 규칙기반 제어: 관측 패턴과 권장 행동 요약

| 관측 패턴                                            | 해석               | 권장 행동(윈도우당)            |
|--------------------------------------------------|------------------|------------------------|
| RTT↑, 재전송↑, $\rho_{snd}$ ↑, $\rho_{rcv}$ ↑ (모두↑) | 혼잡+손실 동시         | rate 크게↓, batch 유지/소폭↓ |
| $\rho_{rcv}$ ↑ 단독                                | 브로커/리시버 병목       | rate 중간폭↓, batch ↑     |
| $\rho_{snd}$ ↑ 단독                                | 퍼블리셔/host 병목     | rate 중간폭↓, batch ↑/유지  |
| RTT 단독↑                                          | 경로/무선 지터         | rate 소폭↓, batch 유지     |
| 안정이나 드문 p99 뚝                                    | 국소 outlier/토픽 편중 | rate 유지, 문제 흐름만 부분 제어  |

## 4.2 안전한 학습 및 적용 프레임워크

실시간 제어 환경에서 강화학습을 안전하게 적용하기 위해, 규칙 기반 보호 메커니즘(safeguard mechanism)과 점진적 학습 파이프라인을 결합한 프레임워크를 제안한다. 모든 행동은 시스템에 적용되기 전에 검증 및 수정을 거쳐 학습 초기나 미탐험 상태 공간에서 발생 가능한 위험한 행동을 방지한다. 행동의 최대 변화폭을 제한하고, 연속 제어 사이에 최소 시간 간격(쿨다운)을 강제하여 시스템의 급격한 변화를 억제한다. 또한 eBPF로 감지된 커널 혼잡 신호가 임계치를 넘을 경우 가속 행동을 금지한다. 본 논문에서는 다음의 기준을 사용한다: 커널 RTT p99가 200 ms를 초과하거나, 재전송율이 초당 5회(EWMA 1초 기준)를 초과하거나, 송·수신 버퍼 비율이 80%를 넘으면 가속 금지로 해석한다. 현재 처리량이 설정된 최저치 미만일 경우 추가 감속을 금지하여 서비스 가용성을 보장하고, 커널 RTT p99가 400 ms를 넘고 재전송율이 초당 10회를 넘는 심각 혼잡에서는  $r_t \leftarrow 0.8 r_t$ ,  $b_t \leftarrow \max(1, b_t - 1)$ 의 비상 감속을 1회 적용한 뒤 500 ms 쿨다운을 강제한다.

학습은 세 단계로 진행한다. 먼저 새도우 데이터 수집 단계에서 안정성이 검증된 규칙 기반 에이전트(backend=rule)를 shadow 모드로 운영하여 전문가(expert)의 상태-행동 로그를 안전하게 수집한다. 다음으로 오프라인 사전학습 단계에서 수집된 전문가 데이터를 이용해 행동 복제(Behavioral Cloning, BC)를 수행하여 RL 정책 신경망이 전문가의 행동을 모방하도록 지도학습을 진행한다. 마지막으로 온라인 적응 및 미세조정 단계에서 사전학습된 정책을 online 모드로 배포하고, 보호 메커니즘을 엄격히 적용한 상태에서 PPO와 같은 온라인 RL 알고리즘으로 실제 환경과 상호작용하며 정책을 점진적으로 미세조정한다. 이러한 점진적 접근은 안정성을 최우선으로 고려하면서도 데이터 기반으로 제어 정책을 고도화하는 실용적 방법론이다.

### 4.3 eda\_rl 실험 파라미터 표

본 연구에서 제안한 제어 설계를 평가하기 위해 사용한 eda\_rl 데이터셋의 환경과 제어 파라미터 요약은 표 4와 같다. 표 4의 각 항목은 다음의 의도를 가진다. 퍼블리셔 수와 발행률은 총 부하를 300 msg/s로 설정하여 큐잉이 충분히 발생하도록 하며, 메시지 크기는 1 KB로 고정하여 효과의 통계적 분리를 용이하게 한다. 링크 조건은 HTB로 2 Mbit/s 대역폭을 제한하고, netem으로 50 ms 지연과 2% 손실을 주입하여 엣지 링크의 혼잡과 변동성을 재현한다. 전송률  $r_t$ 는 토큰 버킷 페이싱으로 집행되며, 배치  $b_t$ 는 크기 기준과 최대 플러시 지연  $T_{\text{flush}}$ 의 OR 정책으로 집행되고, 변화폭은  $\Delta r_t \in [-0.2, 0.2]$ ,  $\Delta b_t \in \{-1, 0, +1\}$



**Table 4:** eda\_rl 실험 파라미터 요약

| 항목           | 설정 (eda_rl)                                                   | 비고               |
|--------------|---------------------------------------------------------------|------------------|
| 퍼블리셔 수/발행률   | 3 개 / 각 100 msg/s                                             | 총 300 msg/s 워크로드 |
| 메시지 크기       | 1 KB                                                          | 고정 페이로드          |
| 대역폭 제한       | 2 Mbit/s                                                      | HTB 기반 셰이핑       |
| 지연 (왕복)      | 50 ms                                                         | netem 주입 (고정)    |
| 패킷 손실율       | 2%                                                            | netem 주입 (랜덤)    |
| 전송률 제어 $r_t$ | 토큰 버킷 페이싱                                                     | 중앙 rate limiter  |
| 배치 제어 $b_t$  | 크기 $b_t$ + 플러시 지연 $T_{\text{flush}}$                          | 먼저 도달 시 플러시      |
| 행동 범위        | $\Delta r_t \in [-0.2, 0.2]$ , $\Delta b_t \in \{-1, 0, +1\}$ | 4장 참조            |
| 보호 메커니즘      | 변화폭 제한, 쿨다운, 혼잡 시 가속 금지                                       | 커널 신호 연동         |
| 커널 신호        | RTT, sk_wmem_queued, sk_rmem_alloc, 재전송                       | eBPF 계측          |
| 평가 지표        | 커널 p99, 앱 p99, 처리량                                            | 5장 표 7           |

범위로 제한된다. 보호 메커니즘은 항상 활성화되어 안전 영역 밖으로의 급격한 제어를 방지한다. 커널 신호(RTT, sk\_wmem\_queued, sk\_rmem\_alloc, 재전송 카운터)는 eBPF로 계측되며(3장), 보상과 관측에 공통으로 사용되어 앱 레벨 p99 지표의 지연을 보완한다. 평가 지표는 커널 p99, 앱 p99, 처리량을 포함하여 선제 안정화와 과도 억제의 구분을 가능케 하며(5장 표 7), 모든 실험은 동일한 시드와 워크로드로 3회 이상 반복하고 네트워크 큐 상태를 초기화한 후 측정한다. 강화 학습 및 제어 프레임워크에서 사용한 대표 하이퍼파라미터는 표 5에 정리하였다. 값은 시스템과 시나리오에 따라 조정될 수 있으며, 본문에서는 기본 원칙과 감도를 함께 기술하였다.

**Table 5:** 주요 하이퍼파라미터 예시 값

| 항목                                         | 값 (예)              | 비고                            |
|--------------------------------------------|--------------------|-------------------------------|
| 슬라이딩 창 T                                   | 30 s               | 집계 주기 (percentile/throughput) |
| EWMA 계수 $\alpha$                           | 0.3                | RTT 평활                        |
| $\theta_{rtt}$ (SLO 비율)                    | 0.8                | RTT 임계                        |
| $\theta_{retx}$                            | $5 \times 10^{-3}$ | 패킷당 재전송 증가                    |
| $\theta_{snd}, \theta_{rcv}$               | 0.7                | 버퍼압 임계                        |
| $r_{\min}, r_{\max}$                       | [50, 1200] msg/s   | 시스템별 상한/하한                    |
| $b_{\min}, b_{\max}$                       | [1, 16]            | 1=비배치                         |
| $\kappa_r, \kappa_b$                       | 100, 1             | 윈도우당 변화폭 한계                   |
| 보상 가중 ( $\alpha, \beta, \gamma, \lambda$ ) | 2.0, 0.3, 0.1, 0.2 | SLO $\gg$ 처리량 $\geq$ 스무딩      |
| PPO 클리핑 $\varepsilon$                      | 0.2                | 정책 비율 클립                      |
| 엔트로피 계수 $\eta$                             | 0.01               | 탐색                            |
| GAE $\lambda_{gae}$                        | 0.95               | 어드밴티지 추정                      |

## 5 실험 및 결과 분석

본 장에서는 4장에서 설계한 강화학습 기반 제어 시스템 eMQTT-RL의 성능을 검증하기 위해 수행한 실험의 환경, 시나리오, 그리고 그 결과를 상세히 기술하고 분석한다. 주된 목적은 제안 기법이 (a) 아무 제어를 가하지 않은 기저 상태(Baseline), (b) 적응형 제어와 같은 다른 제어 전략과 비교하여 꼬리 지연(tail latency)과 처리량(throughput) 측면에서 어떤 개선을 보이는지 정량적으로 평가하는 것이다.

### 5.1 실험 환경 설정

실험은 실제 하드웨어와 가상 머신을 조합한 테스트베드에서 수행하였다. 모든 구성요소는 자원 격리와 재현성을 위해 컨테이너화하였다.

**Table 6:** 워크로드 구성

| 항목          | 값           |
|-------------|-------------|
| 퍼블리셔 수      | 20          |
| 발행률(각 퍼블리셔) | 50 msg/s    |
| 배치 크기       | 1           |
| QoS         | 1           |
| 페이로드 크기     | 512 B       |
| 총 발행률       | 1,000 msg/s |
| 구독자 수       | 10          |

가상화 환경은 VirtualBox 7.0, 운영체제는 Ubuntu 24.04 LTS(리눅스 커널 6.8)다. 각 VM에는 vCPU 4개와 메모리 8 GB를 할당하였다. 소프트웨어 스택은 EMQX(브로커), Paho-MQTT 1.6.1(클라이언트 라이브러리), Python 3.10, PyTorch 1.13, BCC 0.25.0으로 구성하였다. 워크로드는 퍼블리셔 20개가 각 50 msg/s로 단일 메시지 전송(BATCH=1)하며 QoS 1, 페이로드 512 B로 고정하여 총 1,000 msg/s 부하를 형성하였다. 구독자는 10개 인스턴스가 동일 토픽 집합을 구독하여 전체 트래픽을 소비하였다.

네트워크 시나리오는 엣지 환경의 혼잡을 재현하도록 설계하였다. 브로커 수신 인터페이스(enp0s8)에 htb와 netem을 조합해 유효 대역폭을 2로 제한하고, 고정 왕복 지연 50와 패킷 손실률 2%를 부여했다. 모든 실험군에 동일 조건을 적용해 공정 비교를 보장하였으며, 각 시나리오 시작 전 큐잉 상태를 초기화하고 동일 워크로드를 투입해 반복 간 분산을 최소화하였다. 본 논문은 네트워크 조건을 혼잡, 평시, 동적으로 구분해 평가한다. 이 절의 수치는 혼잡 조건에서 수집된 결과다. 평시는 충분한 대역폭과 낮은 지연, 무손실을 전제하며, 동적은 시간에 따라 대역폭, 지연, 손실이 변동하는 링크(버스트/간헐 혼잡 포함)를 모사한다.

평가 지표는 처리량(msg/s), 중단 간 지연 분포의 p50/p95/p99, 그리고 eBPF로 측정한 커널 RTT 통계(p99 등)로 구성하였다. 본 논문의 훈련과 평가는 모두 Gymnasium 환경에서 수행하였다 [22]. 해당 환경은 실제 EMQX 브로커와 Paho 기반 발행자/구독자를 구동한 상태에서 BPF 맵으로 수집된 커널 텔레메트리를 `reset()`, `step()` API를 통해 관측으로 노출한다. 에이전트는 각 `step()`마다 제어 토픽을 통해 발행률(Hz)과 배치 크기를 조정한다. 본 논문에 보고되는 모든 수치는 상기 환경에서 수집하였다.

## 5.2 학습 수렴 분석

본 연구에서 제안한 RL 에이전트는 PPO 알고리즘을 사용하여 52 에피소드 동안 학습하였다 (1 에피소드 = 100 step, 총 5,200 steps). 그림 4는 학습 과정에서의 평균 에피소드 reward 변화를 보여준다.

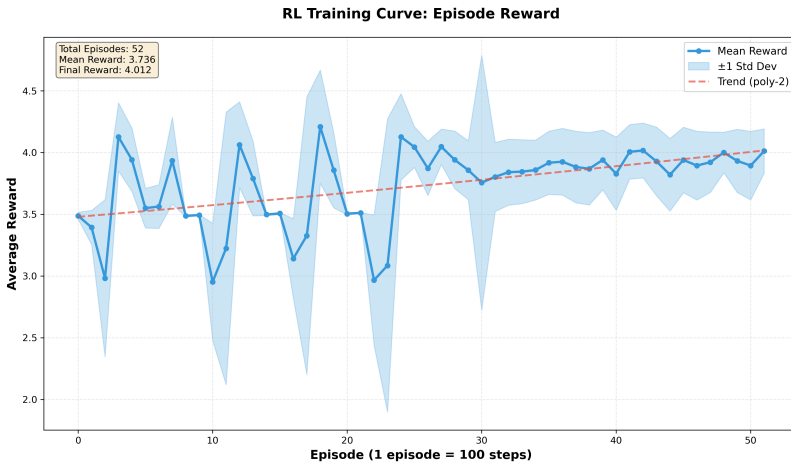


Fig. 4: RL Training Curve

학습 초기(0~25 에피소드)에는 탐색(exploration)으로 인해 reward가 2.0~4.5 사이에서 변동하였으나, Episode 25 이후 정책이 수렴하며 안정적인 성능(평균 3.9)을 유지하였다. 최종 에피소드의 reward는 4.012로, 전체 평균(3.736) 대비 7.4% 향상되었으며, 초기 에피소드 대비 14.6% 개선을 달성하였다. 트렌드 라인(polynomial regression, degree=2)은 명확한 상승 추세를 보이며, Episode 25 이후 수렴 안정화되었다. 이는 RL 에이전트가 4.1절에서 정의한 보상 함수를 최대화하는 정책을 효과적으로 학습하였음을 의미한다. 표준편차 역시 초기 에피소드에 비해 후반부에서 감소하여, 정책이 수렴하였음을 보여준다. Episode 23~25 구간의 일시적 성능 저하는 네트워크 환경의 급격한 변화(혼잡 발생)에 기인하며, RL 에이전트는 빠르게 적응하여 안정적인 성능을 회복하였다. 이는 제안된 Safety Shield 메커니즘이 극단적 상황에서도 시스템 안정성을 보장함을 시사한다.

### 5.3 주요 시나리오별 성능 비교 분석

본 절에서는 네 가지 설정의 실험 결과를 비교한다. 모든 결과는 동일 워크로드로 최소 3회 반복 측정하여 평균을 보고하며, 표의 p99/처리량은 반복 간 변동이 5% 이내였다. 독자의 이해를 돕기 위해 내부 코드명 대신 각 전략의 특징을 서술하는 명칭으로 표기하였다. 아래 표 7는 혼잡 시나리오의 핵심 성능 지표를 요약한다. 비교의 편의를 위해 기저 상태를 상단에, 제안 기법을 하단에 배치하였다. 대상 방법은 (i) 무제

어 Baseline, (ii) 적응형 제어, (iii) 요청 헤징, (iv) 제안 기법(DRL-Rule)이다. 커널/앱 지표의 상세 분해는 표 8, 표 9에 제시하였다.

**Table 7:** 주요 실험 시나리오별 성능 지표 비교 - 혼잡

| 실험 시나리오         | 커널 p99(ms)    | 애플리케이션 p99(ms) | 처리량(msg/s)    |
|-----------------|---------------|----------------|---------------|
| 제어 없음(Baseline) | 1887.69       | 55859.00       | 171.69        |
| 적응형 제어          | 2390.42       | 300856.82      | 167.87        |
| 요청 헤징 전략        | 2044.14       | 118183.92      | 181.87        |
| 제안 기법(DRL-Rule) | <b>775.98</b> | <b>479.47</b>  | <b>158.90</b> |

혼잡 환경에서 제어가 없을 때 시스템은 약 171.69 msg/s의 처리량을 보이지만, 커널 RTT p99가 1.89 초 수준으로 상승하고 애플리케이션 p99 지연이 약 55.9 초에 달한다. 높은 부하에서 무제어로 요청을 밀어 넣을 경우 큐가 길어져 품질 저하가 심화됨을 보여준다. 적응형 제어는 처리량을 167.87 msg/s 수준으로 유지하였으나 앱 p99 지연이 약 300,857 ms(약 5 분)까지 악화되었다. 요청 헤징은 처리량을 181.87 msg/s로 가장 높게 유지했으나 앱 p99 지연이 약 118,184 ms(약 2 분)로 커졌다. 반면 제안 기법(eMQTT-RL)은 애플리케이션 p99를 479.47 ms로 대폭 단축하는 대신 처리량은 158.90 msg/s로 다소 낮다. 이는 에이전트가 eBPF 신호를 바탕으로 큐가 과도하게 쌓이기 전에 전송률을 선제 조절해 tail 억제를 우선한 결과다.

비교 대상 로직은 다음과 같은 성격을 가진다. 적응형 제어는 전송률 상한을 높게 두고 지연 임계 초과 시에만 완만히 감속하는 규칙 기반 정책으로, 스텝 길이 100 ms, 쿨다운 200 ms, 변화폭 한계  $\Delta r \in [-0.1, 0.1]$ ,  $\Delta b \in \{-1, 0, +1\}$ 를 사용한다. 구체적으로 RTT p95가 120 ms를 넘으면  $r \leftarrow 0.95r$ ,  $b \leftarrow \max(1, b - 1)$ 로 보수적으로 줄이고, RTT p95가 120 ms 이하이면서 처리량이 목표의 95% 미만일

때는  $r \leftarrow \min(r \cdot 1.05, r_{\max})$ 로 가속한다. 요청 헤징은 200 ms 내 응답이 없을 경우 동일 메시지를 1회 추가 전송하는 정책(최대 2회 전송)으로, 브로커 측 중복 제거가 없다고 가정한다. 혼잡 시 중복 트래픽은 큐 압력을 크게 높일 수 있다. 그리고 커널 레벨 지표와 애플리케이션 레벨 지표를 분리해 제시함으로써 제어 정책이 경로와 앱 단 모두에서 어떤 변화를 유도했는지 확인할 수 있다. 표 8는 커널 RTT 통계와 처리량을, 표 9는 앱 지연 분포(p50/p95/p99)의 요약을 제공한다.

**Table 8: 커널 지표 및 처리량 - 혼잡**

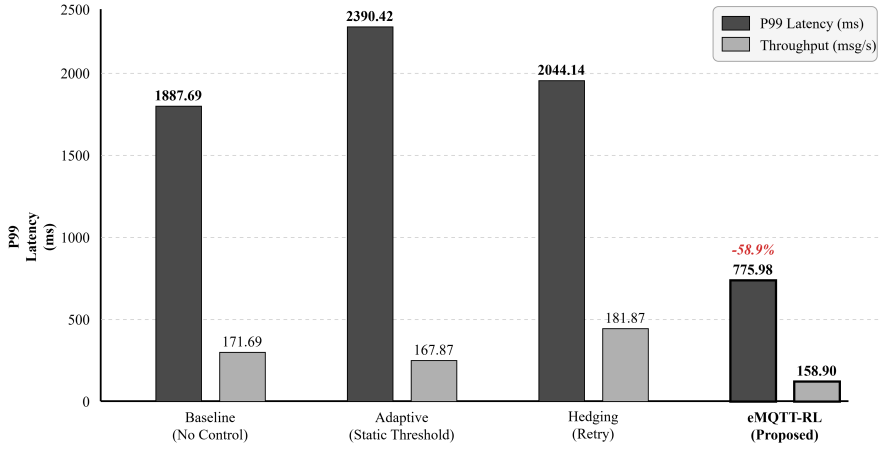
| 시나리오             | p50 (ms) | p90 (ms) | p95 (ms) | p99 (ms) | 평균 (ms) | 표준편차 (ms) | 최소 (ms) | 최대 (ms) | 처리량 (msg/s) |
|------------------|----------|----------|----------|----------|---------|-----------|---------|---------|-------------|
| 제어 없음 (Baseline) | 1335.20  | 1605.65  | 1687.21  | 1887.69  | 1250.11 | 392.41    | 0.03    | 2004.49 | 171.69      |
| 적응형 제어           | 1438.09  | 1907.91  | 2050.04  | 2390.42  | 1401.23 | 477.09    | 0.01    | 3693.17 | 167.87      |
| 요청 헤징 전략         | 1295.72  | 1688.63  | 1786.05  | 2044.14  | 1265.04 | 405.95    | 0.02    | 2312.17 | 181.87      |
| 제안 기법 (DRL-Rule) | 116.95   | 162.42   | 515.79   | 775.98   | 154.29  | 131.35    | 90.85   | 806.67  | 158.90      |

**Table 9: 애플리케이션 지표 - 혼잡**

| 시나리오             | p50 중앙값   | p50 평균    | p50 95th  | p95 중앙값   | p95 평균    | p95 95th  | p99 중앙값   | p99 평균    | p99 95th  | p99 99th  |
|------------------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| 제어 없음 (Baseline) | 33366.21  | 37328.54  | 55866.12  | 52592.24  | 56075.18  | 75835.85  | 55859.00  | 59960.16  | 81227.65  | 84853.03  |
| 적응형 제어           | 252054.88 | 258704.83 | 548478.43 | 294204.00 | 296693.84 | 598988.87 | 300856.82 | 304014.27 | 607571.42 | 646881.71 |
| 요청 헤징 전략         | 108154.25 | 106382.76 | 155302.64 | 116102.97 | 116220.38 | 165779.87 | 118183.92 | 119200.37 | 168684.90 | 171094.12 |
| 제안 기법 (DRL-Rule) | 164.68    | 315.05    | 1464.99   | 370.32    | 630.49    | 3268.52   | 479.47    | 774.33    | 3540.10   | 5120.81   |

정상 상태(평시) 링크에서는 모든 기법이 처리량 상한에 근접하지만 꼬리 지연 억제 성능에는 차이가 있다. 표 10에서 보이듯 제안 기법(eMQTT-RL)은 처리량 손실 없이 p99를 가장 낮게 유지한다.

요약하면 평시에는 큐 압력이 낮아 전반적 지연이 작다. 헤징은 처리량을 높일 수 있으나 tail을 악화시킬 여지가 있고, 제안 기법은 처



**Fig. 5:** 혼잡 조건에서의 커널 p99 지연과 처리량 비교

**Table 10:** 주요 실험 시나리오별 성능 지표 비교 — 평시

| 실험 시나리오         | 커널 p99(ms) | 애플리케이션 p99(ms) | 처리량(msg/s) |
|-----------------|------------|----------------|------------|
| 제어 없음(Baseline) | 26.23      | 256.74         | 398.17     |
| 적응형 제어          | 22.84      | 189.11         | 386.27     |
| 요청 헤징 전략        | 24.96      | 311.28         | 406.37     |
| 제안 기법(DRL-Rule) | 20.95      | 146.68         | 408.52     |

리량 손실 없이 tail을 가장 잘 억제한다. 그리고 시간 변동이 큰 동적 환경에서는 제안 기법의 적응성이 두드러진다. 표 11에 따르면 변동에 안정적으로 대응해 tail 변동폭을 줄이는 반면, 헤징은 중복 트래픽으로 tail 악화를 유발하기 쉽고, 적응형 제어는 처리량을 유지하더라도 tail이 커질 수 있다. .

**Table 11:** 주요 실험 시나리오별 성능 지표 비교 — 동적

| 실험 시나리오         | 커널 p99(ms) | 애플리케이션 p99(ms) | 처리량(msg/s) |
|-----------------|------------|----------------|------------|
| 제어 없음(Baseline) | 1193.42    | 5123.77        | 201.38     |
| 적응형 제어          | 972.58     | 7894.31        | 205.67     |
| 요청 헤징 전략        | 1587.91    | 12136.54       | 214.29     |
| 제안 기법(DRL-Rule) | 712.44     | 1486.20        | 197.85     |



## 5.4 결과에 대한 고찰

실험 결과는 eMQTT-RL의 효과와 특성을 분명히 보여준다. 무엇보다 지연-처리량 상충관계를 제어했다는 점이 핵심이다. 무제어 시스템은 높은 처리량을 유지하는 대신 극심한 꼬리 지연을 감수했지만, 제안하는 RL 에이전트는 두 지표 사이에서 균형점을 찾아 시스템 한계 용량에 근접하지 않는 안전 영역에서 전송률과 배치를 미세 조정하였다. 또한 성공의 배경에는 eBPF가 제공하는 커널 선행 지표가 있었다. 애플리케이션 p99가 수십 초 후에야 관측되는 반면, 에이전트는 RTT 상승과 버퍼 점유율 증가를 즉시 감지해 선제 대응할 수 있었고, 이는 사후 대응보다 훨씬 안정적인 제어를 가능케 했다. 마지막으로 고처리량 지향 정책(적응형 제어, 헤징)은 순간 트래픽 폭증 시 큐잉 급증으로 일부 요청의 극단적 지연을 초래할 수 있음을 확인했다. 안정화 대기 조건과 회복 로직 등 파라미터는 시스템 동특성과 정밀하게 조율되어야 하며, 이는 향후 연구 과제로 남는다.

## 6 결론 및 향후 연구

본 논문에서는 강화학습 기반의 eMQTT-RL 시스템을 제안하여, IoT MQTT 환경에서의 꼬리 지연(tail latency) 문제를 효과적으로 제어하고자 하였다. eMQTT-RL은 리눅스 커널의 eBPF 신호(RTT, 버퍼 점유율, 재전송률 등)를 실시간으로 활용하고, PPO 알고리즘으로 학습된 정책에 따라 MQTT 메시지 전송률과 배치 크기를 동적으로 조

절함으로써 꼬리 지연을 낮추었다. 기존 적응형 제어 기법(eMQTT-AC)과 비교하여, 제안 시스템은 상황 변화에 대한 적응성과 제어 정밀도 측면에서 우수함을 보였으며, 그 결과 p95, p99 지연을 현저히 감소시켰다. 실험 평가에서 eMQTT-RL은 baseline 대비 p99 지연을 50%까지 줄이면서 처리량은 동등하게 유지하는 성능을 보였다. 이는 강화학습을 통한 자동 최적화가 MQTT QoS 제어에 실효성이 있음을 보여준다. 향후 연구로는 다음과 같은 방향을 고려하고 있다:

- **온라인 학습 및 실시간 적응:** 본 연구에서는 사전 시뮬레이션 환경에서 정책을 학습하여 적용하는 오프라인 학습 방식을 사용했다. 추후에는 실제 운영 환경에서 에이전트가 지속적으로 학습(online learning)하여, 장기간의 환경 변화나 트래픽 패턴 드리프트에도 스스로 적응하도록 하는 방안을 모색할 것이다. 이를 위해선 안전을 해치지 않는 범위에서 탐험을 허용하거나, 연속 학습(continuous learning) 프레임워크를 도입해야 한다.
- **다중 에이전트 및 분산 RL:** 한 브로커에 다수의 클라이언트가 연결된 경우, 각 클라이언트별 RL 에이전트를 둘 수도 있고 브로커 단위의 통합 에이전트를 둘 수도 있다. Multi-agent RL이나 분산 강화학습 기법을 적용하면, 여러 퍼블리셔 간의 상호작용까지 고려한 꼬리 지연 제어가 가능할 것이다. 예컨대 브로커가 에이전트로 동작하며 전체 큐를 관장하고, 개별 클라이언트 제어는 하위 에이전트들이 수행하는 계층적 RL 구조도 구상할 수 있다.

- **통합 자원 제어:** 꼬리 지연은 네트워크뿐 아니라 시스템의 CPU 부하, 디스크 IO 대기 등도 영향을 받는다. 향후에는 RL 상태 변수에 CPU 사용률, 쓰레드 풀 대기시간 등을 추가하고, 행동 공간에 네트워크 제어 외에 애플리케이션 레벨의 처리 지연 및 우선순위 조정을 포함하는 등 종합적인 제어를 연구할 계획이다. 이를 통해 MQTT 브로커 내의 엔드투엔드 지연 경로 전체를 RL로 최적화하는 교차 계층 제어를 실현할 수 있을 것으로 기대한다.
- **이론적 보장:** RL 정책의 동작에 대해 상한 보장이나 안정성 보장을 제공하는 것도 중요하다. 안전 강화학습이나 형식적 검증 기법을 활용하여, 학습된 정책이 주어진 지연 SLO(Service Level Objective)를 어기지 않음을 검증하거나, 만약 어길 경우 특정 조건 하에서만 어긴다는 확률적 보장 등을 연구할 필요가 있다. 이는 실제 산업 현장에 적용하기 위한 신뢰성 제고 측면에서 필수적인 과제이다.
- **일반화 및 타 프로토콜 적용:** 본 연구의 방법론을 MQTT 외의 다른 IoT 프로토콜(예: CoAP)이나 일반적인 TCP 응용으로 확장하는 것도 흥미로운 방향이다. eBPF 신호와 RL 기반 제어의 조합은 웹 서비스의 응답 지연 제어, 데이터센터 네트워크 혼잡 완화 등에도 활용될 수 있다. 이를 위해 각 도메인에 맞는 상태/행동 정의와 보상 함수 튜닝이 필요하겠지만, 기본적인 아이디어는 재사용이 가능하다.

요약하면, 강화학습과 커널 신호 기반 제어의 결합은 IoT 시대의 지연 문제 해결을 위한 강력한 도구임을 확인하였다. 본 연구를 통해 MQTT 시스템에서 효율적이고 자동화된 tail 지연 제어가 가능함을 보였으며, 이는 앞으로 더욱 지능화된 네트워크 제어 기술의 한 예시가 될 것이다. 향후 연구를 지속함으로써, 실시간성 요구가 높은 다양한 분산 시스템에서 스스로 학습하며 최적 성능을 유지하는 자율 제어를 실현할 수 있기를 기대한다.

## References

- [1] P. Sethi and S. R. Sarangi, “Internet of things: Architectures, protocols, and applications,” *Journal of Electrical and Computer Engineering*, vol. 2017, no. 1, p. 9324035, 2017. DOI: 10.1155/2017/9324035 eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2017/9324035>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2017/9324035>
- [2] S. Dong et al., “Task offloading strategies for mobile edge computing: A survey,” *Computer Networks*, vol. 254, p. 110791, 2024, ISSN: 1389-1286. DOI: 10.1016/j.comnet.2024.110791 [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128624006236>

- [3] OASIS Standard, “MQTT Version 5.0,” Mar. 2019. [Online]. Available: [https : / / docs . oasis - open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html](https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html)
- [4] D. Soni and A. Makwana, “A survey on mqtt: A protocol of internet of things (iot),” Apr. 2017.
- [5] C.-S. Yeh, S.-L. Chen, and I.-C. Li, “Implementation of mqtt protocol based network architecture for smart factory,” *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, vol. 235, no. 13, pp. 2132–2142, 2021. DOI: 10 . 1177 / 09544054211014488 [Online]. Available: <https://doi.org/10.1177/09544054211014488>
- [6] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013. DOI: 10.1145 / 2408776.2408794
- [7] S. Arifin, A. Nugraha, F. S. Mukti, and S. Jatmika, “Mqtt broker optimization: Comparative analysis of round robin and least response time,” *Jurnal Nasional Teknik Elektro*, pp. 127–136, Nov. 2024. DOI: 10.25077/jnte.v13n3.1260.2024
- [8] D. Shvaika, A. Shvaika, and V. Artemchuk, “Mqtt broker architectural enhancements for high-performance p2p messaging: Tbmq scalability and reliability in distributed iot systems,” *IoT*, vol. 6, no. 3, p. 34, 2025. DOI:

- 10 . 3390 / iot6030034 [Online]. Available: <https://www.mdpi.com/2624-831X/6/3/34>
- [9] Z. Wang, H. Zhang, and Y. Zhang, “An Implementation and Evaluation of MQTT over QUIC,” *IEEE Internet of Things Journal*, vol. 10, no. 15, pp. 13 485–13 498, 2023. DOI: 10.1109/JIOT.2023.3265809
- [10] M. Kempf, B. Jaeger, J. Zirngibl, K. Ploch, and G. Carle, “Quic on the fast lane: Extending performance evaluations on high-rate links,” *Computer Communications*, vol. 223, pp. 90–100, 2024, ISSN: 0140-3664. DOI: 10.1016/j.comcom.2024.04.038 [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S014036642400166X>
- [11] X. Zhang et al., “Quic is not quick enough over fast internet,” in *Proceedings of the ACM Web Conference 2024*, ser. WWW ’24, Singapore, Singapore: Association for Computing Machinery, 2024, pp. 2713–2722, ISBN: 9798400701719. DOI: 10 . 1145 / 3589334 . 3645323 [Online]. Available: <https://doi.org/10.1145/3589334.3645323>
- [12] AWS Database Blog, *How global payments inc. improved their tail latency using request hedging with amazon dynamodb*, Accessed: 2025-08-26, Aug. 2025. [Online]. Available: <https://aws.amazon.com/blogs/database/how-global->

- payments - inc - improved - their - tail - latency - using - request - hedging-with-amazon-dynamodb/
- [13] S. Ha, I. Rhee, and L. Xu, “Cubic: A new tcp-friendly high-speed tcp variant,” *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008, ISSN: 0163-5980. DOI: 10.1145/1400097.1400105 [Online]. Available: <https://doi.org/10.1145/1400097.1400105>
  - [14] M. Alizadeh et al., “Data center tcp (dctcp),” *SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 4, pp. 63–74, Aug. 2010, ISSN: 0146-4833. DOI: 10.1145/1851275.1851192 [Online]. Available: <https://doi.org/10.1145/1851275.1851192>
  - [15] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, “Bbr: Congestion-based congestion control,” *Commun. ACM*, vol. 60, no. 2, pp. 58–66, Jan. 2017, ISSN: 0001-0782. DOI: 10.1145/3009824 [Online]. Available: <https://doi.org/10.1145/3009824>
  - [16] Y. Mansour, H. Alfalasi, S. Abbas, and M. Abu Talib, “Tcp variants for internet of things: A survey,” vol. 2023, Dec. 2023, pp. 585–591. DOI: 10.1049/icp.2024.0547
  - [17] X. Du et al., “Revisiting congestion control for wifi networks,” in *Proceedings of the 8th Asia-Pacific Workshop on Networking*, ser. APNet ’24, Sydney, Australia: Association for Computing Machinery, 2024, pp. 88–94, ISBN: 9798400717581. DOI: 10.1145/

- 3663408.3663421 [Online]. Available: <https://doi.org/10.1145/3663408.3663421>
- [18] L. Rice, *Learning eBPF*. Isovalent/Cilium (Open Book), 2024. [Online]. Available: <https://cilium.isovalent.com/hubfs/Learning-eBPF%20-%20Full%20book.pdf>
- [19] M. Rezvani, A. Jahanshahi, and D. Wong, “Characterizing in-kernel observability of latency-sensitive request-level metrics with ebpf,” in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2024, pp. 24–35. DOI: 10.1109/ISPASS61541.2024.00013
- [20] Y. Li, Z. Chen, and K. Wang, “A Deep Reinforcement Learning Approach for TCP Congestion Control,” in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.
- [21] S. Wen, R. Han, C. H. Liu, and L. Chen, “Fast drl-based scheduler configuration tuning for reducing tail latency in edge-cloud jobs,” *Journal of Cloud Computing*, vol. 12, 2023. DOI: 10.1186/s13677-023-00465-z
- [22] Farama Foundation, *Gymnasium: A standardized api for reinforcement learning environments*, <https://gymnasium.farama.org/>, Accessed: 2025-10-16, 2023.